HomemadePickles

An AWS Cloud-Powered E-Commerce Platform for Artisan Food Products

SmartBridge Educational Services | SmartInternz

Cloud Computing Internship Project

March 2026

Homemade Pickles and Snacks

Project Description

Homemade Pickles is a full-stack, cloud-based e-commerce platform designed to sell artisan homemade pickles, snacks, and traditional food products online. Built using Flask as the backend framework, the application is deployed on Amazon EC2 and leverages DynamoDB for efficient, scalable storage of user data and order records.

The platform enables customers to register, log in, browse products across three categories — Non-Veg Pickles, Veg Pickles, and Snacks — add items to a shopping cart, and complete their purchases through a smooth checkout process. Orders are stored in real time in AWS DynamoDB, enabling reliable order tracking and data persistence without the overhead of a relational database.

By combining the scalability of AWS cloud infrastructure with a clean, intuitive Flask-powered interface, Homemade Pickles delivers a seamless shopping experience for customers who value traditional, handcrafted food products. The platform is hosted on an EC2 instance (Amazon Linux 2023) and uses IAM roles to securely authenticate access to AWS services.

Scenario 1: User Registration & Secure Login

A new customer visits the Homemade Pickles website and creates an account by providing their username, email address, and password. The Flask backend securely hashes the password using Werkzeug before storing it in the DynamoDB Users table. On subsequent visits, the user logs in with their credentials — the system retrieves their hashed password from DynamoDB, verifies it, and establishes an authenticated session. The entire flow ensures that user data is never stored in plain text.

Scenario 2: Browse Products & Add to Cart

Once logged in, a customer can explore the three product categories: Non-Veg Pickles (Chicken Pickle, Mutton Pickle, Gongura Prawns, etc.), Veg Pickles (Mango Pickle, Tomato Pickle, etc.), and Snacks (Banana Chips, Chekkalu, Dry Fruit Laddu, etc.). Each product is available in multiple weight options (250g, 500g, 1000g) with corresponding prices. Items selected by the customer are managed via Flask session state, allowing them to build and review their cart before checkout.

Scenario 3: Checkout & Order Storage in DynamoDB

At checkout, the customer provides their delivery details (name, address, phone number) and selects a payment method. The Flask backend validates all inputs, assigns a unique UUID-based order ID, and stores the complete order — including product items, total amount, and timestamp — in the DynamoDB Orders table. This real-time data storage ensures every transaction is reliably persisted, enabling future order management and reporting features.

AWS Architecture

Homemade Pickles utilises a lean, cost-effective AWS architecture centred around EC2 for compute and DynamoDB for managed NoSQL storage, with IAM handling secure role-based access.

AWS Service	Role	Details
Amazon EC2	Application Hosting	t2.micro, Amazon Linux 2023
Amazon DynamoDB	NoSQL Database	Users & Orders tables
AWS IAM	Access Control	EC2 role with DynamoDB access
Flask (Python)	Backend Framework	Routes, Auth, Cart, Checkout
GitHub	Version Control	Source code repository

Architecture Overview

The web browser communicates with the Flask application running on an EC2 instance over HTTP (port 5000). The Flask app interacts with two DynamoDB tables — Users (for authentication) and Orders (for order persistence) — using the boto3 SDK. An IAM role attached to the EC2 instance grants the necessary permissions to perform DynamoDB operations without hard-coded credentials.

Entity Relationship (ER) Diagram

The following diagram illustrates the relationship between the Users and Orders tables in DynamoDB. A single user can place multiple orders (one-to-many relationship).

Users Table	Relationship	Orders Table
username (PK)	1 to Many	order_id (PK)
email		username (FK)

password		name address
		phone items (JSON)
		total_amount payment_method timestamp

Pre-Requisites

- AWS Account Setup — Active AWS account with billing configured
- Understanding IAM — IAM roles, policies, and EC2 instance profiles
- Amazon EC2 Basics — Launching instances, security groups, key pairs
- DynamoDB Basics — Table creation, partition keys, put/get operations
- Python & Flask — Flask routing, sessions, Jinja2 templates
- Git Version Control — Cloning repositories, pushing code changes
- boto3 SDK — AWS SDK for Python to interface with DynamoDB

Project Workflow

1. AWS Account Setup and Login

- Activity 1.1: Set up an AWS account and configure billing settings.
- Activity 1.2: Log in to the AWS Management Console.

2. DynamoDB Database Creation and Setup

- Activity 2.1: Navigate to DynamoDB in the AWS Console.
- Activity 2.2: Create Users and Orders tables with appropriate partition keys.

3. Backend Development and Application Setup

- Activity 3.1: Develop the Flask backend (app.py) with routes for auth, products, cart, and checkout.
- Activity 3.2: Implement DynamoDB integration using boto3 for user and order operations.
- Activity 3.3: Set up setup_db.py to programmatically create DynamoDB tables.

4. IAM Role Setup

- Activity 4.1: Create an IAM Role for EC2 to access DynamoDB.
- Activity 4.2: Attach the AmazonDynamoDBFullAccess managed policy.

5. EC2 Instance Setup

- Activity 5.1: Launch an EC2 instance (t2.micro, Amazon Linux 2023).
- Activity 5.2: Configure security groups for SSH (22) and application port (5000).
- Activity 5.3: Attach the IAM role to the EC2 instance.

6. Deployment on EC2

- Activity 6.1: Connect to EC2 using EC2 Instance Connect.
- Activity 6.2: Clone the GitHub repository and install dependencies.
- Activity 6.3: Run the Flask application.

7. Testing and Verification

- Activity 7.1: Conduct functional testing of user registration, login, cart, and checkout.
- Activity 7.2: Verify DynamoDB data updates after each user operation.

Milestone 1: AWS Account Setup and Login

Activity 1.1: Set up an AWS Account

Sign up for an AWS account at aws.amazon.com and configure billing alerts to monitor usage and stay within free-tier limits.

Activity 1.2: Log in to the AWS Management Console

After account setup, log in to the AWS Management Console using root user credentials. For day-to-day operations, it is best practice to use IAM users with limited permissions.

Milestone 2: DynamoDB Database Creation and Setup

Activity 2.1: Navigate to DynamoDB

In the AWS Management Console, search for DynamoDB and navigate to the Tables section. Click Create table to begin provisioning a new table.

Activity 2.2: Create the Users and Orders Tables

Two DynamoDB tables are required for the HomemadePickles application:

Users Table

- Table Name: Users
- Partition Key: username (String)
- Billing Mode: On-Demand (or Provisioned with 5 RCU / 5 WCU)

Orders Table

- Table Name: Orders
- Partition Key: order_id (String)
- Billing Mode: On-Demand (or Provisioned with 5 RCU / 5 WCU)

These tables are also programmatically created using the setup_db.py script:

```
import boto3

def create_tables():
    dynamodb = boto3.client('dynamodb', region_name='ap-south-1')
    existing_tables = dynamodb.list_tables()['TableNames']

    if 'Users' not in existing_tables:
        dynamodb.create_table(
            TableName='Users',
            KeySchema=[{'AttributeName': 'username', 'KeyType': 'HASH'}],
            AttributeDefinitions=[{'AttributeName': 'username', 'AttributeType': 'S'}],
            ProvisionedThroughput={'ReadCapacityUnits': 5, 'WriteCapacityUnits': 5}
        )

    if 'Orders' not in existing_tables:
        dynamodb.create_table(
            TableName='Orders',
            KeySchema=[{'AttributeName': 'order_id', 'KeyType': 'HASH'}],
            AttributeDefinitions=[{'AttributeName': 'order_id', 'AttributeType': 'S'}],
            ProvisionedThroughput={'ReadCapacityUnits': 5, 'WriteCapacityUnits': 5}
```

```
        )

    waiter = dynamodb.get_waiter('table_exists')
    waiter.wait(TableName='Users')
    waiter.wait(TableName='Orders')
    print("Tables are active.")
```

The following screenshot shows the DynamoDB tables (Users and Orders) in Active status after creation:

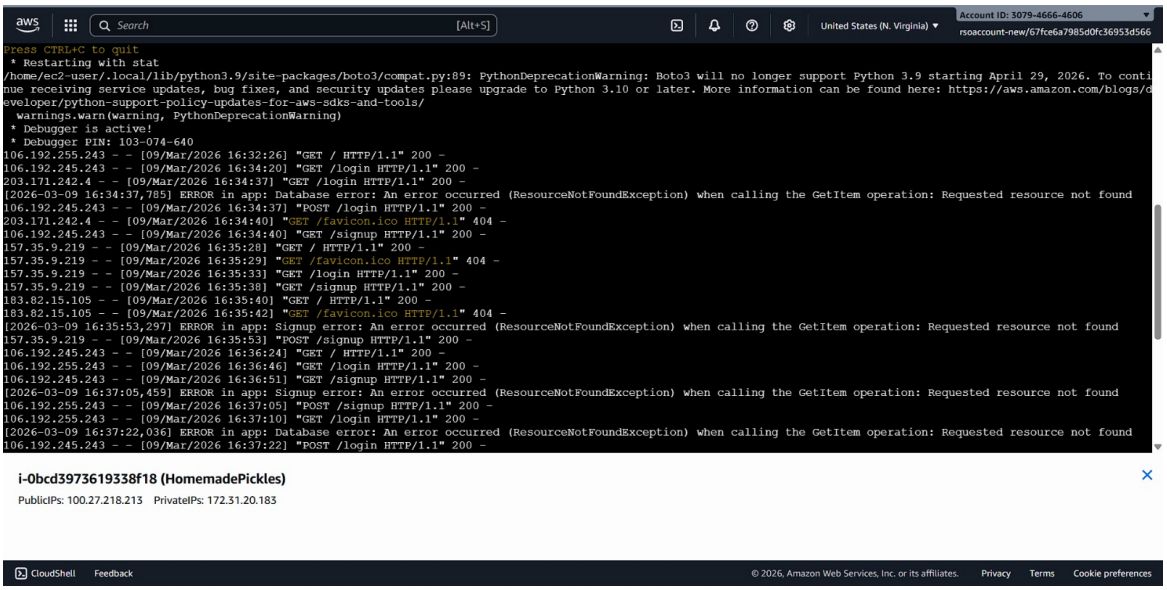


Figure 1: DynamoDB Tables — Users and Orders in Active Status

Milestone 3: Backend Development and Application Setup

Activity 3.1: Flask Application Structure

The HomemadePickles Flask application is organised with HTML templates in a templates/ directory, static assets (CSS, images) in static/, and the main application logic in app.py. The project is managed on GitHub at: https://github.com/havinashpatil/Aws-Project/

Activity 3.2: Flask App — Key Code Sections

Import Libraries & Initialisation

```
from flask import Flask, render_template, request, redirect, url_for, session
from werkzeug.security import generate_password_hash, check_password_hash
import boto3
from datetime import datetime
import json, uuid
from decimal import Decimal

app = Flask(__name__)
app.secret_key = 'your_very_secret_key_12345'

# DynamoDB connection
dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')
users_table = dynamodb.Table('Users')
orders_table = dynamodb.Table('Orders')
```

User Registration Route

The signup route collects username, email, and password, checks for duplicate usernames in DynamoDB, hashes the password using Werkzeug, and stores the new user record.

```
@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        username = request.form['username'].strip()
        email    = request.form['email'].strip()
        password = request.form['password']
        response = users_table.get_item(Key={'username': username})
        if 'Item' in response:
            return render_template('signup.html', error='Username already exists')
        hashed = generate_password_hash(password)
        users_table.put_item(Item={'username': username, 'email': email, 'password': hashed})
        return redirect(url_for('login'))
    return render_template('signup.html')
```

User Login Route

The login route fetches the user record from DynamoDB, verifies the submitted password against the stored hash, and initialises the session on success.

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        response = users_table.get_item(Key={'username': username})
        if 'Item' not in response:
            return render_template('login.html', error='User not found')
        user = response['Item']
        if check_password_hash(user['password'], password):
            session['logged_in'] = True
            session['username'] = username
            return redirect(url_for('home'))
        return render_template('login.html', error='Invalid password')
    return render_template('login.html')
```

Product Catalogue

Products are defined as an in-memory dictionary with three categories. Each product has an id, name, and a dictionary mapping weight options (250g, 500g, 1000g) to prices in INR.

```
products = {
    'non_veg_pickles': [
        {'id': 1, 'name': 'Chicken Pickle',    'weights': {'250': 600, '500': 1200, '1000':
1800}},
        {'id': 2, 'name': 'Fish Pickle',       'weights': {'250': 200, '500': 400,  '1000':
800}},
        {'id': 3, 'name': 'Gongura Mutton',    'weights': {'250': 400, '500': 800,  '1000':
1600}},
        # ... more products
    ],
    'veg_pickles': [
        {'id': 7, 'name': 'Traditional Mango Pickle', 'weights': {'250': 150, '500': 280,
'1000': 500}},
        # ... more products
    ],
    'snacks': [
        {'id': 7,  'name': 'Banana Chips',    'weights': {'250': 300, '500': 600, '1000':
800}},
        {'id': 13, 'name': 'Dry Fruit Laddu', 'weights': {'250': 500, '500': 1000, '1000':
1500}},
        # ... more products
    ]
}
```

Checkout & Order Storage Route

The checkout route validates all form inputs, parses the cart JSON, converts the total to Decimal for DynamoDB compatibility, and stores a complete order record with a UUID-based order ID.

```
@app.route('/checkout', methods=['GET', 'POST'])
def checkout():
    if request.method == 'POST':
        name           = request.form.get('name', '').strip()
        address        = request.form.get('address', '').strip()
        phone          = request.form.get('phone', '').strip()
        payment_method = request.form.get('payment', '').strip()
        cart_items     = json.loads(request.form.get('cart_data', '[]'))
```

```
        total_amount   = Decimal(request.form.get('total_amount', '0'))
        orders_table.put_item(Item={
            'order_id':       str(uuid.uuid4()),
            'username':       session.get('username', 'Guest'),
            'name':           name,
            'address':        address,
            'phone':          phone,
            'items':          cart_items,
            'total_amount':   str(total_amount),
            'payment_method': payment_method,
            'timestamp':      datetime.now().isoformat()
        })
        return redirect(url_for('sucess'))
    return render_template('checkout.html')
```

Milestone 4: IAM Role Setup

Activity 4.1: Create IAM Role

In the AWS Console, navigate to IAM → Roles and click Create role. Select AWS service as the trusted entity type and choose EC2 as the use case. This allows the EC2 instance to assume the role and call AWS APIs on your behalf.

Activity 4.2: Attach Policies

Attach the following managed policy to the role:

- AmazonDynamoDBFullAccess — Grants the EC2 instance full read/write access to all DynamoDB tables in the account, enabling the Flask app to store and retrieve user and order data without requiring hard-coded credentials.

The screenshot below shows the IAM Roles list with the StudentUser role successfully created, alongside existing service-linked roles:

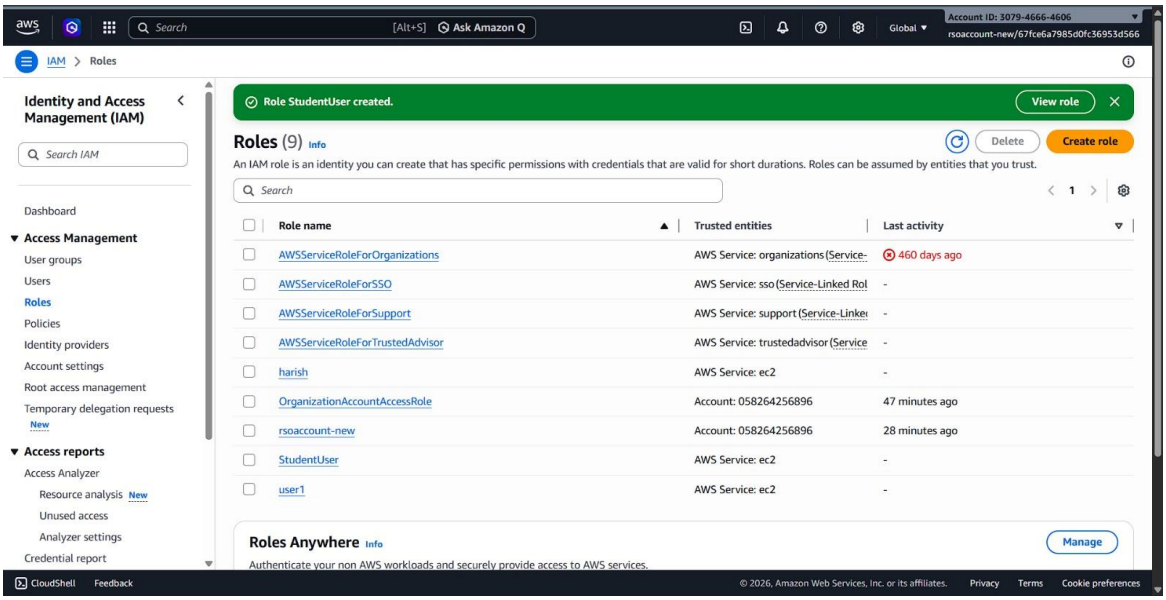


Figure 2: IAM Roles — StudentUser Role Created Successfully

The following screenshot shows the attempted permissions boundary setup and the access-denied error, which was resolved by adjusting the IAM permissions on the management account:

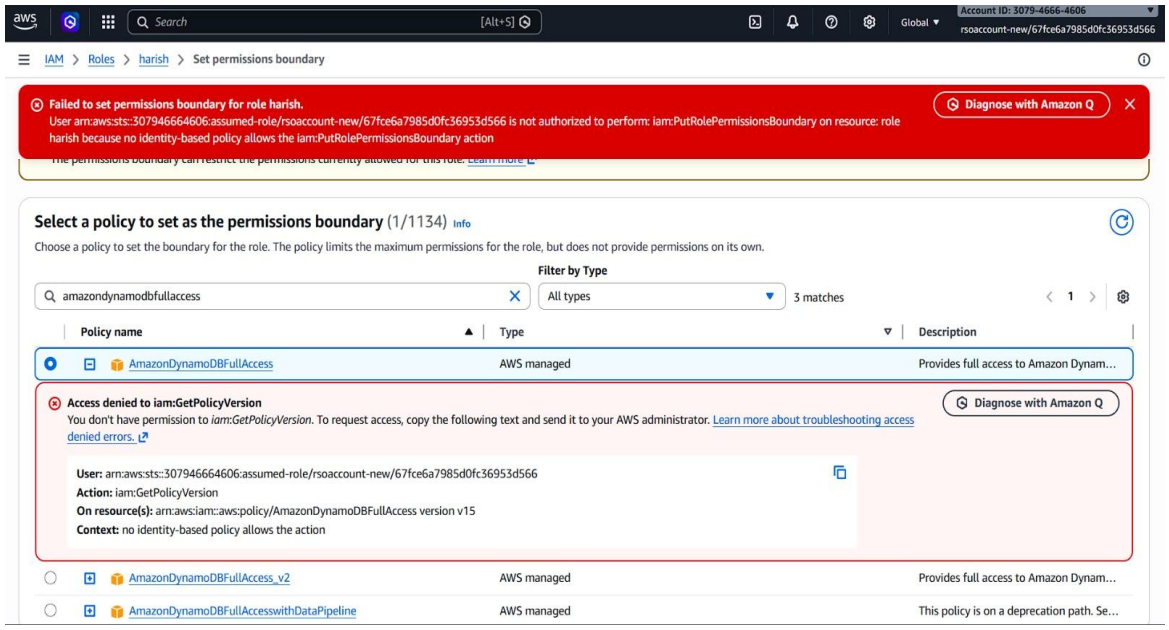


Figure 3: IAM Permissions Boundary Configuration

Milestone 5: EC2 Instance Setup

Activity 5.1: Launch EC2 Instance

Navigate to EC2 in the AWS Console and click Launch instances. Configure the instance with the following settings:

- Name: HomemadePickles
- AMI: Amazon Linux 2023
- Instance Type: t2.micro (Free Tier eligible)
- Key Pair: Create or use an existing key pair for SSH access
- Storage: 8 GiB gp3 EBS volume

Activity 5.2: Configure Security Groups

Add the following inbound rules to the security group:

- SSH — TCP port 22 (for secure terminal access)
- Custom TCP — port 5000 (for Flask application access via browser)
- HTTP — TCP port 80 (optional, for standard web traffic)

Activity 5.3: Attach IAM Role

After launching the instance, attach the IAM role created in Milestone 4 by selecting the instance → Actions → Security → Modify IAM Role. This grants the Flask application on EC2 permission to access DynamoDB without hard-coded credentials.

The screenshot below shows the HomemadePickles EC2 instance in Running state with 2/2 status checks passing and its public IP address:

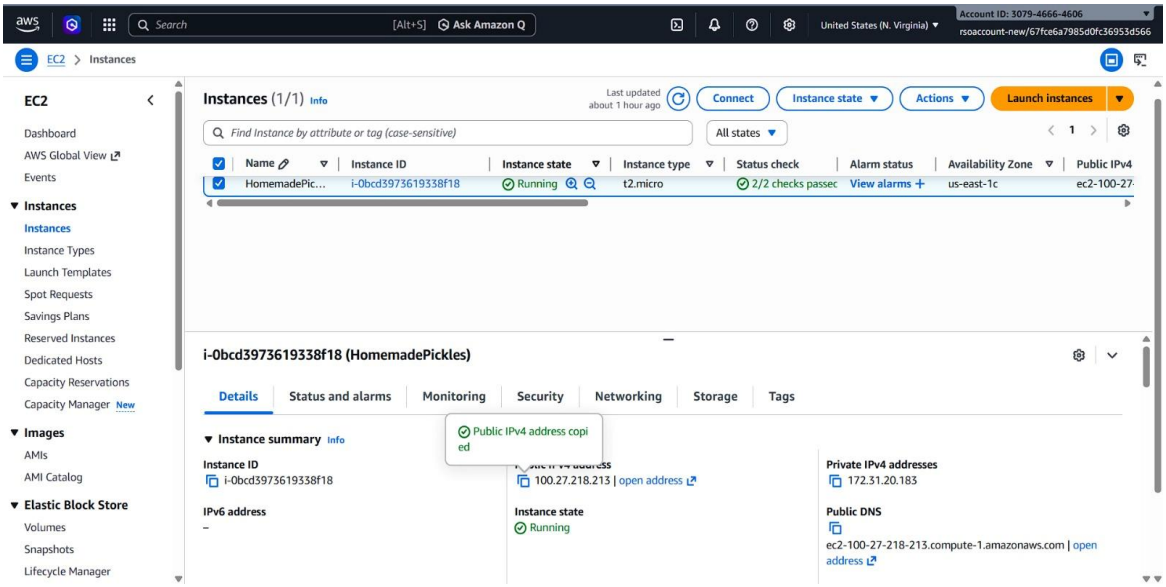


Figure 4: EC2 Instance — HomemadePickles Running (t2.micro, us-east-1c)

Milestone 6: Deployment on EC2

Activity 6.1: Connect to EC2 via EC2 Instance Connect

In the EC2 Console, select the HomemadePickles instance and click Connect. Choose EC2 Instance Connect to open a browser-based terminal without needing a local SSH client.

Activity 6.2: Install Dependencies & Clone Repository

Run the following commands in the EC2 terminal to set up the environment:

```
# Update packages and install Git
sudo yum update -y
sudo yum install git -y

# Clone the GitHub repository
git clone https://github.com/havinashpatil/Aws-Project
cd Aws-Project/homemade-main

# Install Python dependencies
pip install flask boto3 werkzeug
```

Activity 6.3: Run the Flask Application

Start the Flask application using Python 3. The app runs on all interfaces (0.0.0.0) on port 5000 in debug mode:

```
# Run the application
python3 app.py

# Access via browser:
# http://<EC2-Public-IP>:5000
```

The screenshot below shows the EC2 terminal with the Flask application running after cloning from GitHub:

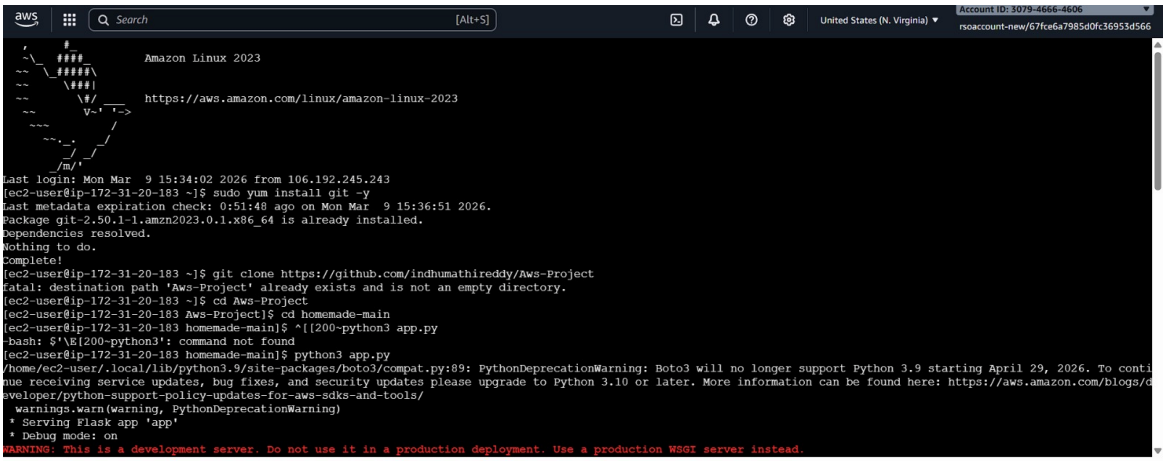


Figure 5: EC2 Terminal — Flask App Running (python3 app.py)

Milestone 7: Testing and Deployment

Activity 7.1: Application Screenshots

User Registration Page

The registration form collects the username, email, and password. On submission, the data is stored securely in the DynamoDB Users table with a bcrypt-hashed password.

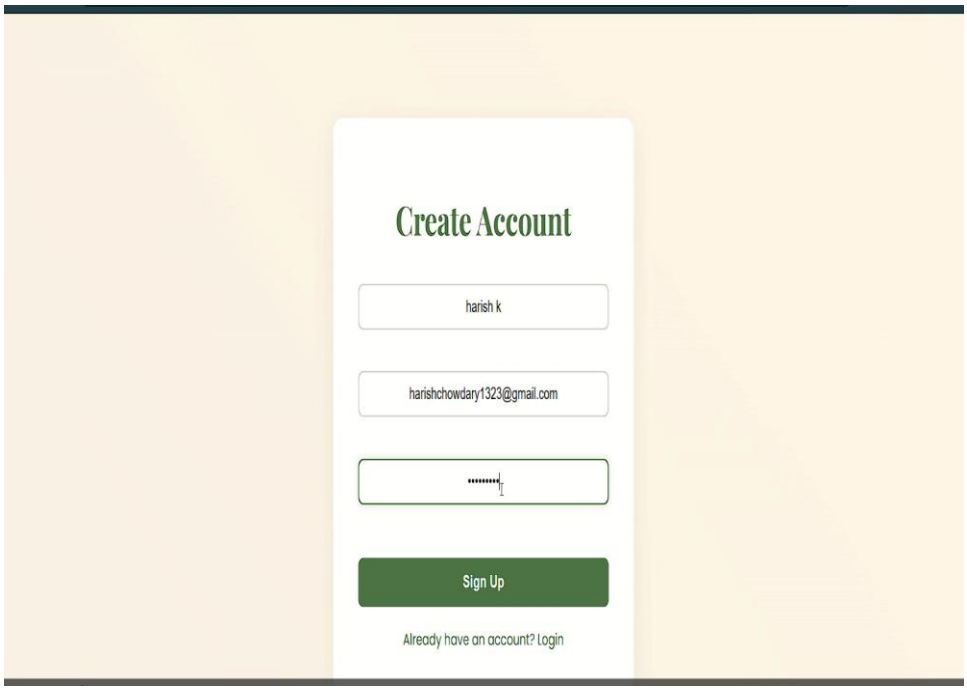
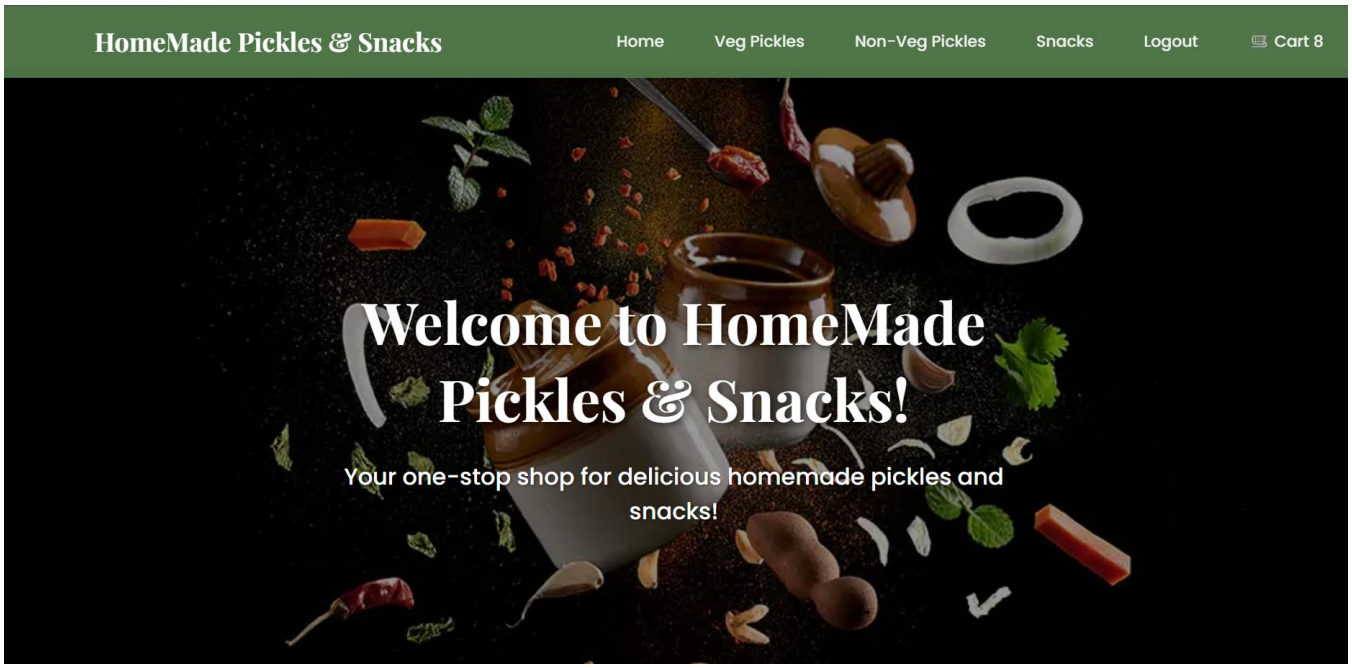
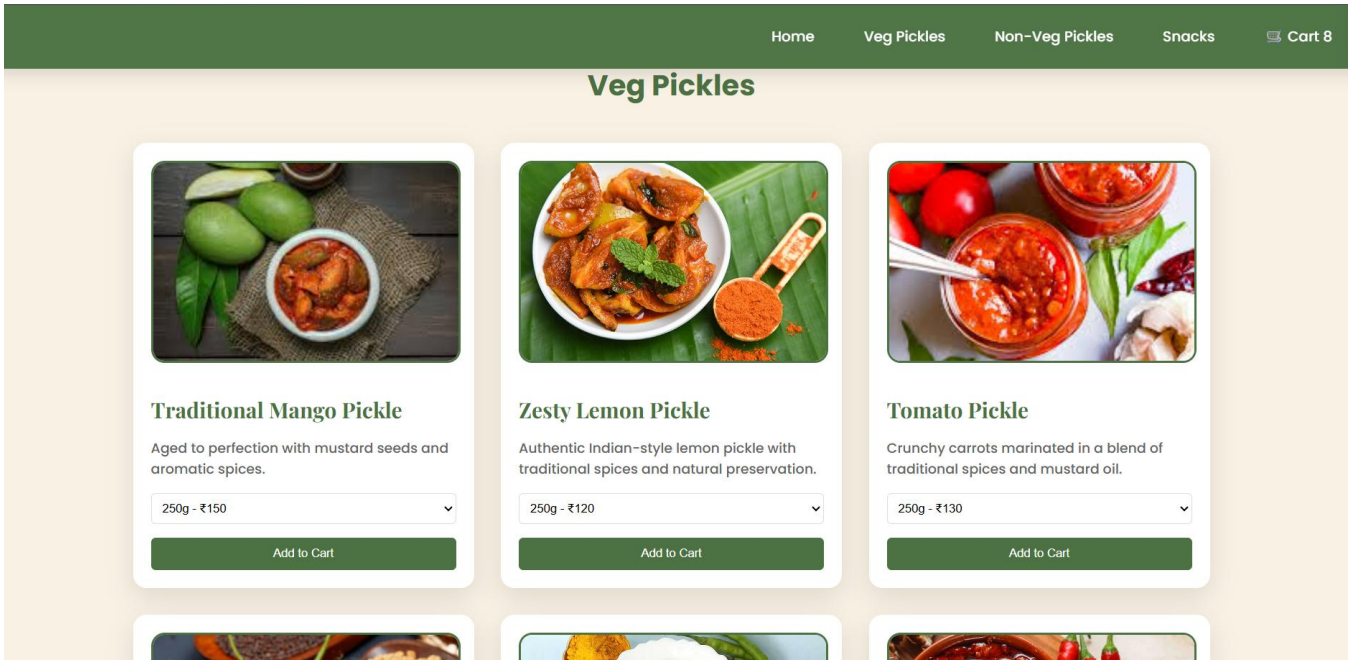


Figure 6: Create Account (Signup) Page

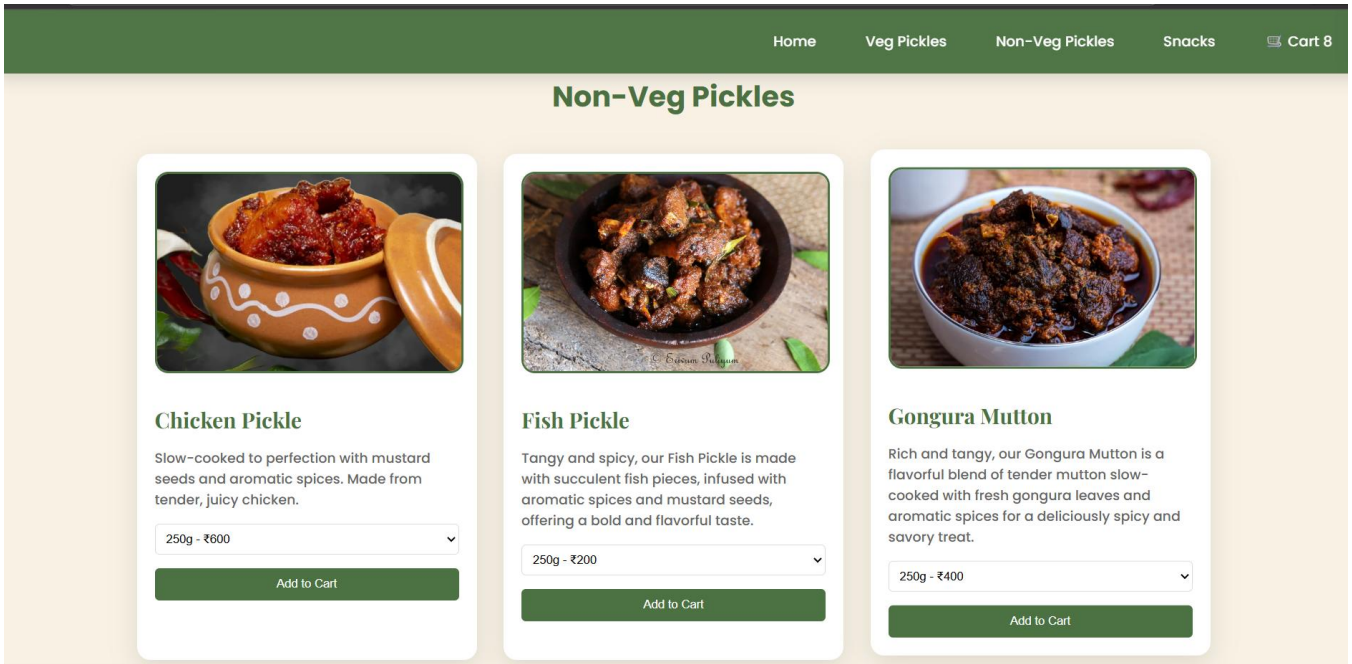
Home Page :



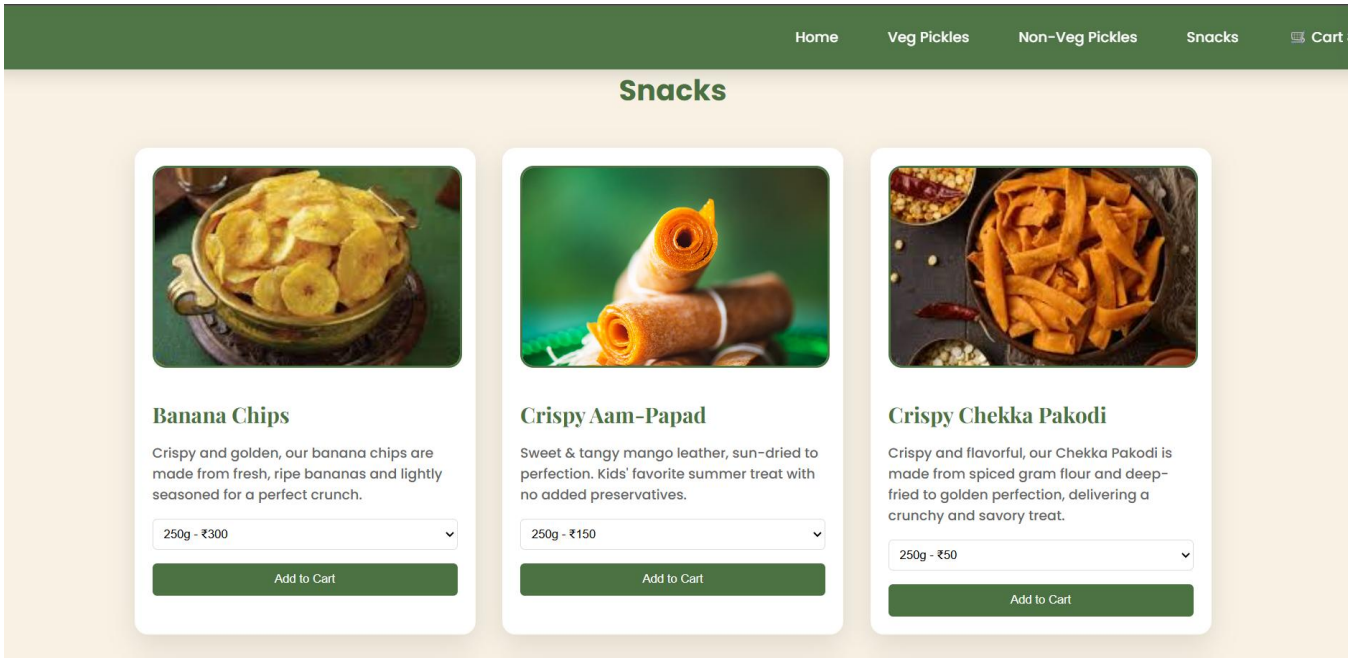
Veg Pickles :



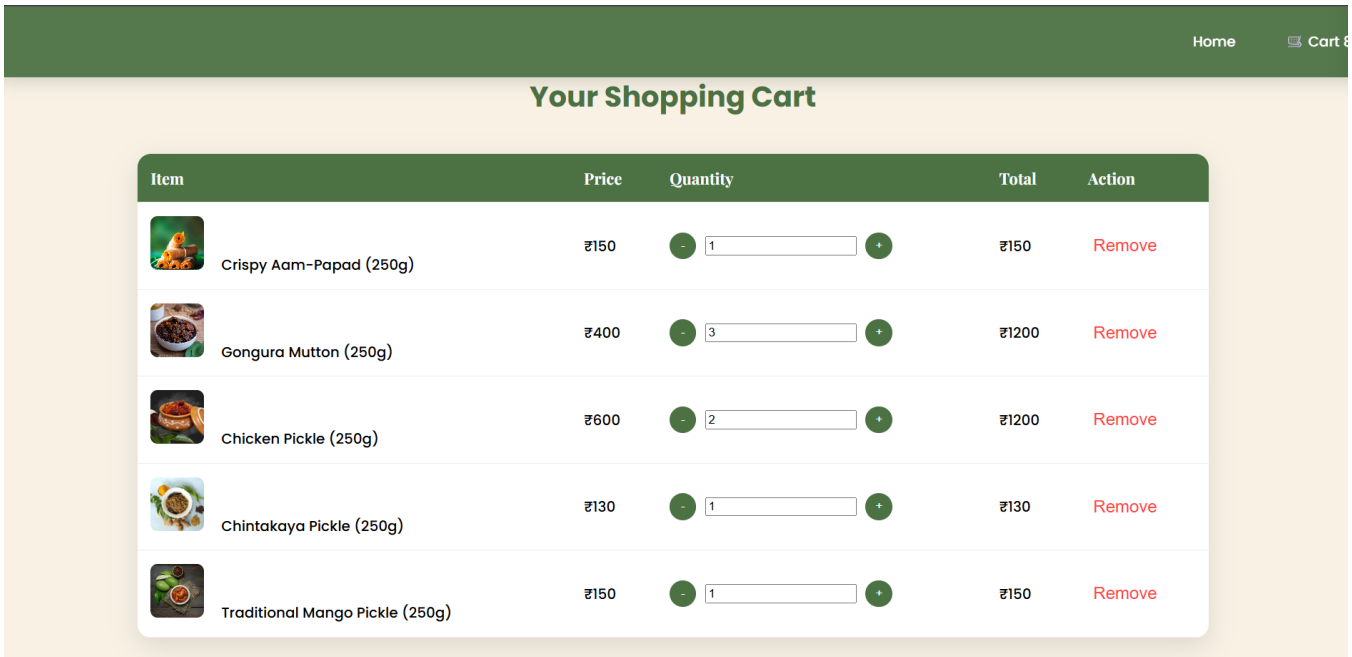
Non-Veg Pickles :



Snacks :

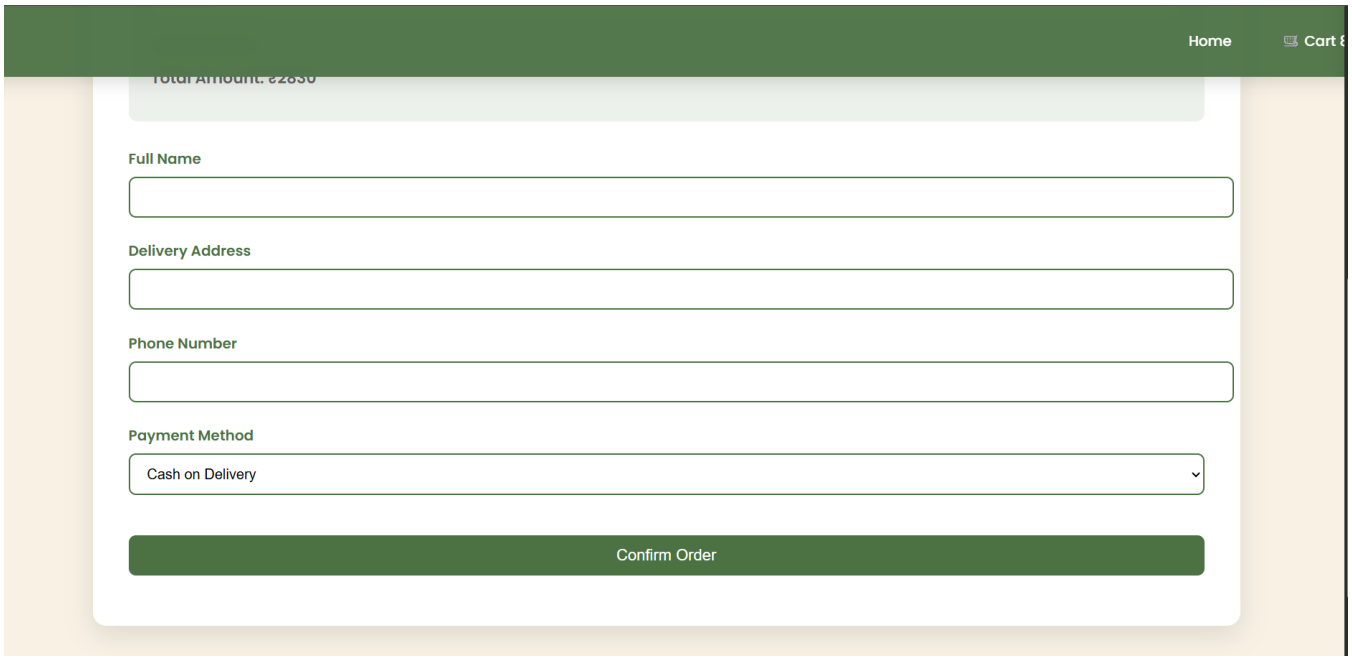


Cart :



Checkout :

Delivery Address :



Flask Application Logs

The following screenshot shows live Flask application logs from EC2 CloudShell, demonstrating successful GET/POST requests to the login, signup, and home routes. The ResourceNotFoundException errors for early requests indicate the DynamoDB tables were not yet provisioned in the correct region, which was subsequently resolved.

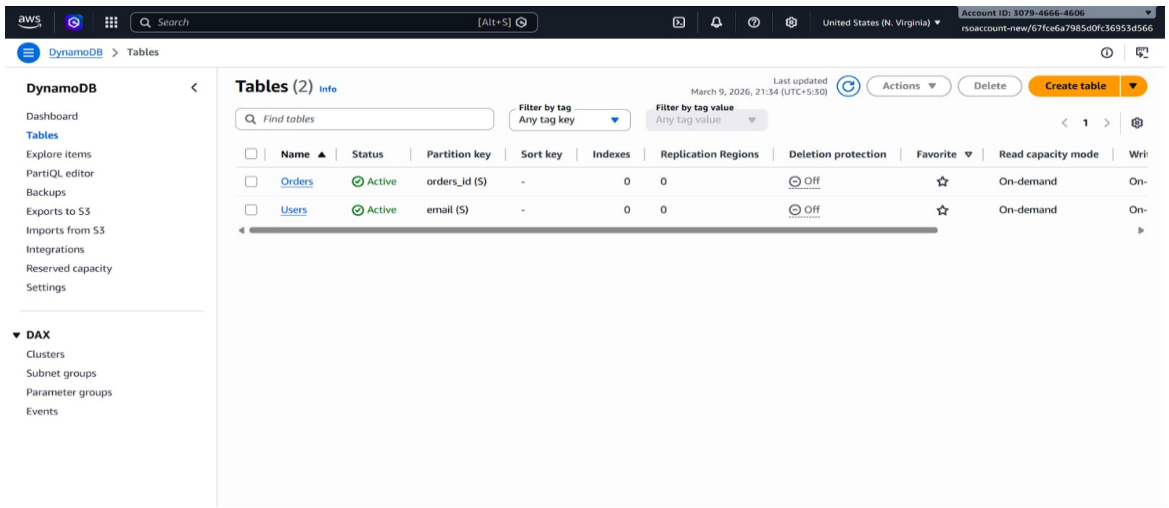


Figure 7: Flask Application Logs — EC2 CloudShell

DynamoDB Database Updates

Users Table

The Users table stores registered user records. Each item contains the username (partition key), email address, and bcrypt-hashed password. The following screenshot shows the DynamoDB table structure and confirms the tables are Active:

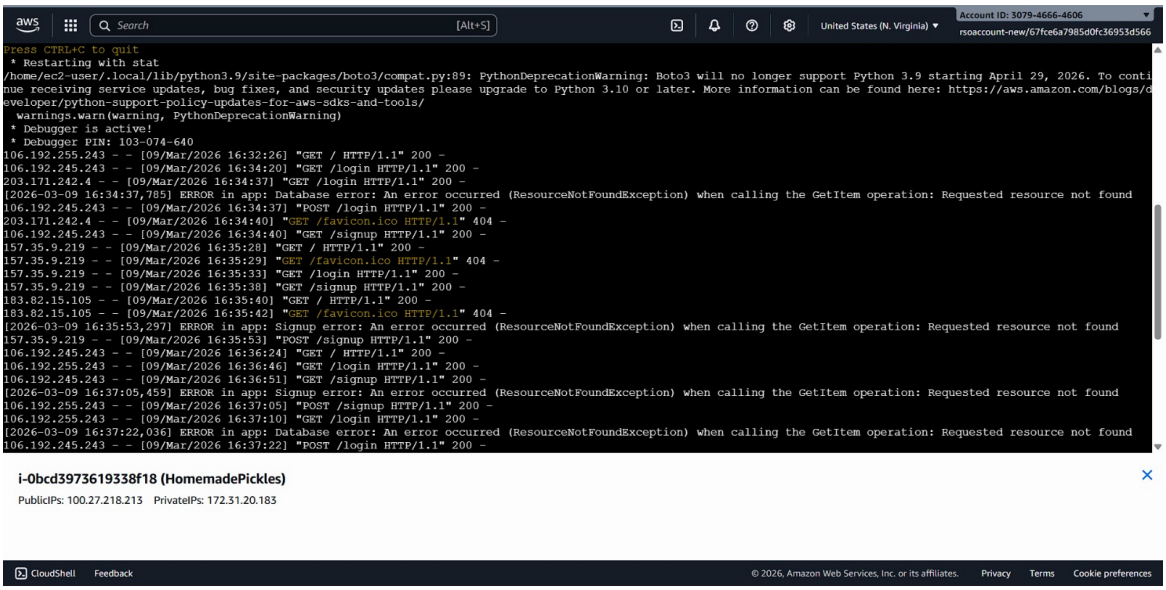


Figure 8: DynamoDB Tables — Users and Orders in Active Status

Orders Table

The Orders table stores every order placed through the checkout flow. Each record contains a UUID-based order_id, the customer's username, name, delivery address, phone number, the JSON array of cart items, total amount, payment method, and a timestamp. The on-demand billing mode ensures the table scales automatically with traffic.

Table Schema Summary

Table	Field	Type	Notes
Users	username	String (PK)	Partition key
Users	email	String	User email address
Users	password	String	bcrypt hashed

Orders	order_id	String (PK)	UUID v4
Orders	username	String	FK → Users
Orders	items	List (JSON)	Cart items array
Orders	total_amount	String	Decimal → str
Orders	timestamp	String	ISO 8601 format

Conclusion

The Homemade Pickles e-commerce platform has been successfully developed and deployed using a robust, cloud-native architecture on AWS. By leveraging Amazon EC2 for scalable application hosting and Amazon DynamoDB for managed, serverless data storage, the platform delivers a reliable and performant shopping experience for customers seeking traditional, handcrafted food products.

The Flask backend handles all core operations efficiently — user registration with password hashing, session-based authentication, product catalogue management across three categories, shopping cart management, and secure order processing with DynamoDB persistence. IAM roles ensure that AWS service access is governed by the principle of least privilege, with no hard-coded credentials in the application code.

The deployment pipeline — from GitHub repository to a running EC2 instance — demonstrates a practical cloud deployment workflow that can be extended with features such as HTTPS via an Application Load Balancer, CI/CD pipelines, SNS email notifications on order placement, and S3-hosted product images.

In summary, Homemade Pickles showcases the power of combining Flask's simplicity with AWS's managed cloud services to build a production-ready e-commerce solution that is cost-effective, scalable, and maintainable.